

Machine Learning Assignment: Inventory Demand Forecasting

Assignment Overview

Course: Machine Learning Fundamentals: Regression

Maximum Points: 100

Problem Statement

Business Context

RetailMart Inc. is a mid-sized retail chain operating across multiple regions with diverse product categories. The company is facing significant challenges with inventory management:

1. **Stockouts:** Products frequently run out of stock, leading to lost sales and customer dissatisfaction
2. **Overstock:** Excess inventory ties up capital and increases warehousing costs
3. **Spoilage/Obsolescence:** Perishable and seasonal items often expire or become obsolete

Your Role: You have been hired as a Data Scientist to build a machine learning model that predicts the **optimal number of units to stock** for each product-store combination. Your model should help the company:

- Minimize stockout incidents by at least 30%
- Reduce excess inventory costs by 20%
- Improve overall inventory turnover ratio

Business Objective

"Develop a predictive model to estimate the exact number of units to stock for each product at each store location, considering historical sales patterns, seasonality, promotions, and external factors."

Dataset Description

You are provided with **5 CSV files** representing a dimensional data model (star schema):

Fact Table

File	Description
fact_sales.csv	Daily sales transactions with quantities, revenue, and inventory levels

Dimension Tables

File	Description
dim_products.csv	Product master data (category, subcategory, price, supplier)
dim_stores.csv	Store information (location, size, type, region)
dim_dates.csv	Calendar dimension (dates, holidays, seasons, weekends)
dim_suppliers.csv	Supplier information (lead time, reliability score)

Data Quality Issues (Intentional - You Must Clean!)

The data contains realistic quality issues that you must address:

- Missing values in various columns
- Inconsistent date formats
- Duplicate records
- Typos in categorical variables (e.g., "Electroncs" vs "Electronics")
- Outliers in sales quantities
- Mismatched keys between tables
- Inconsistent casing (e.g., "NORTH", "North", "north")

Assignment Tasks

Part 1: Data Preparation (25 Points)

Task 1.1: Data Loading & Initial Assessment (5 points)

- Load all 5 CSV files
- Display basic information (shape, dtypes, head)
- Document the data quality issues you find

Task 1.2: Data Cleaning (10 points)

- Handle missing values appropriately (document your strategy)
- Fix inconsistent categorical values
- Remove or handle duplicates
- Correct data type issues
- Handle outliers (document your approach)

Task 1.3: Data Integration (10 points)

- Join all dimension tables with the fact table
- Create a single flat file for analysis
- Validate the joins (check for data loss or duplication)
- Export the cleaned flat file as `cleaned_inventory_data.csv`

Part 2: Exploratory Data Analysis (20 Points)

Answer the following business questions with visualizations and insights:

Sales Analysis Questions

1. What are the top 10 best-selling products by quantity and revenue?
2. Which product categories have the highest and lowest sales?
3. How do sales vary across different store types and regions?

Time-Based Analysis Questions

4. What are the seasonal patterns in sales? (Monthly/Quarterly trends)
5. How do sales differ between weekdays and weekends?

6. **What is the impact of holidays on sales volume?**

Inventory Analysis Questions

7. **Which products have the highest stockout frequency?**

8. **What is the correlation between promotional activities and sales?**

9. **How does supplier lead time affect inventory levels?**

Advanced Analysis

10. **Create a heatmap showing the relationship between all numerical features**

11. **Identify any interesting patterns or anomalies in the data**

12. **What features do you think will be most important for prediction?**

Part 3: Feature Engineering (15 Points)

Feature engineering is the process of creating new input features from your existing data to help your machine learning model make better predictions. Good features can significantly improve model performance.

Create the following meaningful features for your ML model:

3.1 Lag Features (What happened in the past?)

What are Lag Features?

Lag features capture historical values of a variable. They help the model understand past patterns to predict future demand.

Why use them?

If a product sold 100 units yesterday, it's likely to sell a similar amount today. Past sales are strong indicators of future sales.

Required Lag Features:

- `sales_lag_1` : Previous day's sales
- `sales_lag_7` : Sales from 7 days ago (same day last week)
- `sales_lag_30` : Sales from 30 days ago

Example Code:

```
# Sort data by product and date first
df = df.sort_values(['product_id', 'date_id'])

# Create lag features using shift()
# shift(1) moves values down by 1 row (yesterday's value)
df['sales_lag_1'] = df.groupby('product_id')['quantity_sold'].shift(1)
df['sales_lag_7'] = df.groupby('product_id')['quantity_sold'].shift(7)
df['sales_lag_30'] = df.groupby('product_id')['quantity_sold'].shift(30)
```

Visual Example:

Date	quantity_sold	sales_lag_1	sales_lag_7
Jan 1	50	NaN	NaN
Jan 2	45	50	NaN
Jan 3	60	45	NaN
...	...	...	...
Jan 8	55	48	50

3.2 Rolling Statistics (What's the trend?)

What are Rolling Statistics?

Rolling (or moving) statistics calculate aggregations over a sliding window of time. They smooth out daily fluctuations and reveal underlying trends.

Why use them?

A single day's sales can be noisy (random variation). A 7-day average gives a more stable picture of typical demand.

Required Rolling Features:

- sales_ma_7 : 7-day moving average
- sales_ma_30 : 30-day moving average
- sales_std_7 : 7-day rolling standard deviation (measures volatility)

Example Code:

```
# Rolling mean (moving average)
# window=7 means average of last 7 days
# min_periods=1 allows calculation even with fewer than 7 values
df['sales_ma_7'] = df.groupby('product_id')['quantity_sold'].transform(
    lambda x: x.rolling(window=7, min_periods=1).mean()
)

df['sales_ma_30'] = df.groupby('product_id')['quantity_sold'].transform(
    lambda x: x.rolling(window=30, min_periods=1).mean()
)

# Rolling standard deviation (measures how much sales vary)
df['sales_std_7'] = df.groupby('product_id')['quantity_sold'].transform(
    lambda x: x.rolling(window=7, min_periods=1).std()
)
```

Visual Example:

Date	quantity_sold	sales_ma_7
Jan 1	50	50.0
Jan 2	45	47.5
Jan 3	60	51.7
Jan 4	40	48.8
Jan 5	55	50.0
Jan 6	70	53.3
Jan 7	65	55.0
Jan 8	50	55.0

3.3 Time-Based Features (When does it happen?)

What are Time Features?

Time features extract components from date/time that capture seasonal patterns, weekly cycles, and

special events.

Why use them?

Sales patterns differ by day of week (weekends vs weekdays), month (holiday seasons), and special events (holidays).

Required Time Features:

- `day_of_week` : 0=Monday, 6=Sunday
- `month` : 1-12
- `quarter` : 1-4
- `is_weekend` : 1 if Saturday/Sunday, else 0
- `is_holiday` : 1 if holiday, else 0

Example Code:

```
# Convert date column to datetime if not already
df['date'] = pd.to_datetime(df['date_id'])

# Extract time components
df['day_of_week'] = df['date'].dt.dayofweek      # 0=Monday, 6=Sunday
df['month'] = df['date'].dt.month                # 1-12
df['quarter'] = df['date'].dt.quarter            # 1-4
df['day_of_month'] = df['date'].dt.day           # 1-31
df['week_of_year'] = df['date'].dt.isocalendar().week # 1-52

# Create binary flags
df['is_weekend'] = df['day_of_week'].isin([5, 6]).astype(int) # Sat=5, Sun=6

# is_holiday should come from your dim_dates table after joining
```

3.4 Categorical Encoding (Converting text to numbers)

What is Categorical Encoding?

Machine learning models require numerical inputs. Categorical encoding converts text categories (like "Electronics", "North") into numbers.

Why use it?

Models cannot process text directly. We must convert categories to numerical representations while preserving meaningful information.

Two Main Approaches:

A) Label Encoding - For ordinal categories (with natural order)

```
# Example: Store size has a natural order
size_mapping = {'Small': 1, 'Medium': 2, 'Large': 3}
df['store_size_encoded'] = df['store_size'].map(size_mapping)
```

B) One-Hot Encoding - For nominal categories (no natural order)

```
# Example: Region has no natural order (North is not "greater than" South)
df_encoded = pd.get_dummies(df, columns=['region', 'category', 'store_type'], drop_first=True)

# This creates binary columns like:
# region_South, region_East, region_West, region_Central
# (region_North is dropped as the reference category)
```

Visual Example of One-Hot Encoding:

Before:

product_id	category
P1001	Electronics
P1002	Clothing
P1003	Electronics

After:

product_id	category_Clothing	category_Electronics
P1001	0	1
P1002	1	0
P1003	0	1

3.5 Handling NaN Values from Feature Engineering

After creating lag and rolling features, the first few rows will have NaN values (because there's no "yesterday" for the first day).

```
# Option 1: Drop rows with NaN (loses some data)
```

```
df = df.dropna()
```

```
# Option 2: Fill with appropriate values
```

```
df['sales_lag_1'] = df['sales_lag_1'].fillna(df['quantity_sold']) # Use current value
```

```
df['sales_ma_7'] = df['sales_ma_7'].fillna(df['quantity_sold']) # Use current value
```

Summary: Complete Feature Engineering Pipeline

```
# 1. Sort data
df = df.sort_values(['product_id', 'date_id'])

# 2. Create lag features
df['sales_lag_1'] = df.groupby('product_id')['quantity_sold'].shift(1)
df['sales_lag_7'] = df.groupby('product_id')['quantity_sold'].shift(7)

# 3. Create rolling features
df['sales_ma_7'] = df.groupby('product_id')['quantity_sold'].transform(
    lambda x: x.rolling(7, min_periods=1).mean()
)

# 4. Create time features
df['date'] = pd.to_datetime(df['date_id'])
df['day_of_week'] = df['date'].dt.dayofweek
df['month'] = df['date'].dt.month
df['is_weekend'] = df['day_of_week'].isin([5, 6]).astype(int)

# 5. Encode categorical variables
df_encoded = pd.get_dummies(df, columns=['category', 'region', 'store_type'], drop_first=True)

# 6. Handle missing values
df_encoded = df_encoded.dropna()

# 7. Verify your features
print(df_encoded.columns.tolist())
print(df_encoded.shape)
```

Part 4: Machine Learning Modeling (30 Points)

Task 4.1: Data Preparation for ML (5 points)

- Define your target variable (units_to_stock or demand)
- Split data into training (80%) and testing (20%) sets
- Apply appropriate scaling if needed

Task 4.2: Model Training (15 points)

Train and evaluate the following **5 algorithms**:

#	Algorithm	Points
1	Linear Regression (Baseline)	3
2	Ridge or Lasso Regression	3
3	Decision Tree Regressor	3
4	Random Forest Regressor	3
5	Gradient Boosting Regressor	3

For each model:

- Train the model
- Make predictions on test set
- Calculate evaluation metrics

Task 4.3: Model Evaluation & Comparison (10 points)

Use the following metrics to evaluate all models:

- **RMSE (Root Mean Squared Error)**
- **MAE (Mean Absolute Error)**
- **R² Score (Coefficient of Determination)**
- **MAPE (Mean Absolute Percentage Error)** - Bonus

Create a comparison table and visualizations showing:

- Performance metrics for all 5 models
- Bar chart comparing model performances
- Residual plots for top 2 models
- Feature importance (for tree-based models)

Recommend the best model with justification.

Part 5: Streamlit Application (10 Points)

Build a simple Streamlit web application that allows users to:

1. **Input Parameters:**
- Select a product category
 - Select a store
 - Choose a date
 - Indicate if there's a promotion
2. **Output:**
- Predicted units to stock
 - Confidence interval (if possible)
 - Historical comparison chart
3. **Dashboard Features:**
- Display model performance metrics
 - Show feature importance
 - Basic data visualizations

Hint: Use AI tools (ChatGPT, Claude, etc.) to help generate the Streamlit code. Document which prompts you used.

Deliverables

Submit the following files:

#	File	Description
1	01_data_cleaning.ipynb	Data cleaning and integration notebook
2	02_eda.ipynb	Exploratory Data Analysis notebook
3	03_feature_engineering.ipynb	Feature engineering notebook
4	04_ml_modeling.ipynb	ML modeling and comparison notebook
5	app.py	Streamlit application code
6	cleaned_inventory_data.csv	Cleaned flat file
7	best_model.pkl	Saved best model (using joblib or pickle)

#	File	Description
8	README.md	Project documentation

Hints & Tips

Data Cleaning

- Use `df.info()` , `df.describe()` , `df.isnull().sum()` for initial assessment
- String methods like `.str.lower()` , `.str.strip()` help standardize text
- `pd.to_datetime()` for date parsing

Model Comparison

```
# Example: Results DataFrame
results = pd.DataFrame({
    'Model': ['Linear Regression', 'Ridge', 'Decision Tree', 'Random Forest', 'Gradient Boosting'],
    'RMSE': [rmse_lr, rmse_ridge, rmse_dt, rmse_rf, rmse_gb],
    'MAE': [mae_lr, mae_ridge, mae_dt, mae_rf, mae_gb],
    'R2': [r2_lr, r2_ridge, r2_dt, r2_rf, r2_gb]
})
```

Streamlit App Prompt Example

"Create a Streamlit app for inventory demand prediction. The app should have sidebar inputs for product category (dropdown), store (dropdown), date (date picker), and promotion (checkbox). Display the predicted units to stock and a bar chart showing historical sales. Use a pre-trained model loaded from 'best_model.pkl'."

Good luck! Remember: The goal is to learn, not just to complete the assignment.